Maintenance of Equipments

Table of Contents

- 1. Data Overview
  - 1.1. Importing libraries
  - 1.2. Reading csv file
- 2. Data Preprocessing
  - 2.1. Checking for nulls (missing values) and uniques
  - 2.2. Missing values
  - 2.3. Checking for duplicate rows
  - 2.4. Dropping unnecessary columns (UDI and Product ID)
  - 2.5. Identifying categorical columns
  - 2.6. Value counts for categorical columns
  - 2.7. Identifying faulty machines with 'Target' value 1
  - 2.8. Removing rows with 'No Failure' in faulty machines
  - 2.9. Identifying non-faulty machines with 'Target' value 0
  - 2.10. Removing rows with 'Random Failures' in non-faulty machines
  - 2.11. Replacing infinite values with NaN
  - 2.12. Encoding and mapping categorical variables
  - 2.13. Rename the variables
- 3. Exploratory Data Analysis
  - 3.1. Describing the variables
  - 3.2. Skewness
  - 3.3. Box plot
  - 3.4. Individual box plot
  - 3.5. Histograms
  - 3.5. Pairplot visualization
- 4. Correlation Analysis
  - 4.1. Correlation matrix
- 5. Predictive Modeling
  - 5.1. Determinating X and Y
  - 5.2. Smote (y class balanced)
  - 5.3. Gridseach
  - 5.4. K – Fold Cross Validation
  - 5.5. Train – Test predictive modelling
- 6. Model Evaluation
  - 6.1. Metrics indicators

Importing libraries

```
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.io as pio
import seaborn as sns
import matplotlib.pyplot as plt
pio.templates.default = 'plotly_white'
from sklearn import metrics
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.model_selection import KFold
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
from sklearn.metrics import confusion_matrix
from sklearn import tree
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import classification_report
from sklearn.model_selection import GridSearchCV
```

```
from imblearn.over_sampling import SMOTE
from collections import Counter
```

Reading csv file

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
df=pd.read_csv("/content/drive/My Drive/Colab Notebooks/predictive_maintenance.csv")
```

```
df
```

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Target	Failure Type
0	1	M14860	M	298.1	308.6	1551	42.8	0	0	No Failure
1	2	L47181	L	298.2	308.7	1408	46.3	3	0	No Failure
2	3	L47182	L	298.1	308.5	1498	49.4	5	0	No Failure
3	4	L47183	L	298.2	308.6	1433	39.5	7	0	No Failure
4	5	L47184	L	298.2	308.7	1408	40.0	9	0	No Failure
...	...	...	...	...	...	...	...	...	...	...
9995	9996	M24855	M	298.8	308.4	1604	29.5	14	0	No Failure
9996	9997	H39410	H	298.9	308.4	1632	31.8	17	0	No Failure
9997	9998	M24857	M	299.0	308.6	1645	33.4	22	0	No Failure
9998	9999	H39412	H	299.0	308.7	1408	48.5	25	0	No Failure
9999	10000	M24859	M	299.0	308.7	1500	40.2	30	0	No Failure

10000 rows × 10 columns

Describing the variables

```
df.shape
```

```
(10000, 10)
```

```
df.head(5)
```

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Target	Failure Type
0	1	M14860	M	298.1	308.6	1551	42.8	0	0	No Failure
1	2	L47181	L	298.2	308.7	1408	46.3	3	0	No Failure
2	3	L47182	L	298.1	308.5	1498	49.4	5	0	No Failure
3	4	L47183	L	298.2	308.6	1433	39.5	7	0	No Failure
4	5	L47184	L	298.2	308.7	1408	40.0	9	0	No Failure

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 10 columns):
 #   Column                  Non-Null Count  Dtype  
---  ------                  --------------  -----  
 0   UDI                     10000 non-null  int64  
 1   Product ID              10000 non-null  object 
 2   Type                    10000 non-null  object 
 3   Air temperature [K]     10000 non-null  float64
 4   Process temperature [K] 10000 non-null  float64
 5   Rotational speed [rpm]  10000 non-null  int64  
 6   Torque [Nm]             10000 non-null  float64
 7   Tool wear [min]         10000 non-null  int64  
 8   Target                  10000 non-null  int64  
 9   Failure Type            10000 non-null  object 
dtypes: float64(3), int64(4), object(3)
memory usage: 781.4+ KB
```

```
df.describe()
```

	UDI	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Target
count	10000.00000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	5000.50000	300.004930	310.005560	1538.776100	39.986910	107.951000	0.033900
std	2886.89568	2.000259	1.483734	179.284096	9.968934	63.654147	0.180981
min	1.00000	295.300000	305.700000	1168.000000	3.800000	0.000000	0.000000
25%	2500.75000	298.300000	308.800000	1423.000000	33.200000	53.000000	0.000000
50%	5000.50000	300.100000	310.100000	1503.000000	40.100000	108.000000	0.000000
75%	7500.25000	301.500000	311.100000	1612.000000	46.800000	162.000000	0.000000
max	10000.00000	304.500000	313.800000	2886.000000	76.600000	253.000000	1.000000

Checking for nulls and uniques

```
1 df['Target'].unique()
```

```
array([0, 1])
```

```
1 df['Failure Type'].unique()
```

```
array(['No Failure', 'Power Failure', 'Tool Wear Failure',
       'Overstrain Failure', 'Random Failures',
       'Heat Dissipation Failure'], dtype=object)
```

```
1 df['Type'].unique()
```

```
array(['M', 'L', 'H'], dtype=object)
```

```
1 columns_to_check = ['UDI', 'Product ID', 'Type', 'Air temperature [K]', 'Process temperature [K]', 'Rotational speed [rpm]', 'Torque
2
3 for col in columns_to_check:
4     print(df[df[col].isna()])
```

```
Empty DataFrame
Columns: [UDI, Product ID, Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], Target
Index: []
Empty DataFrame
Columns: [UDI, Product ID, Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], Target
Index: []
Empty DataFrame
Columns: [UDI, Product ID, Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], Target
Index: []
Empty DataFrame
Columns: [UDI, Product ID, Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], Target
Index: []
Empty DataFrame
Columns: [UDI, Product ID, Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], Target
Index: []
Empty DataFrame
Columns: [UDI, Product ID, Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], Target
Index: []
Empty DataFrame
Columns: [UDI, Product ID, Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], Target
Index: []
Empty DataFrame
Columns: [UDI, Product ID, Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], Target
Index: []
Empty DataFrame
Columns: [UDI, Product ID, Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], Target
Index: []
```

Missing values

```
1 null_df = df.isnull().sum() * 100/len(df)
2
3 info_df = pd.DataFrame({'qtd_missing': df.isnull().sum(),
4                         'percent_missing': null_df,
5                         'dtype': df.dtypes}
6                        ).reset_index().rename(columns={'index': 'column'})
7 info_df
```

	column	qtd_missing	percent_missing	dtype
0	UDI	0	0.0	int64
1	Product ID	0	0.0	object
2	Type	0	0.0	object
3	Air temperature [K]	0	0.0	float64
4	Process temperature [K]	0	0.0	float64
5	Rotational speed [rpm]	0	0.0	int64
6	Torque [Nm]	0	0.0	float64
7	Tool wear [min]	0	0.0	int64
8	Target	0	0.0	int64
9	Failure Type	0	0.0	object

Checking for duplicate rows

```
df.duplicated().sum()
```

```
np.int64(0)
```

Dropping unnecessary columns (UDI and Product ID)

```
df.drop(['UDI', 'Product ID'], axis=1, inplace=True)
```

```
df.head()
```

	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Target	Failure Type
0	M	298.1	308.6	1551	42.8	0	0	No Failure
1	L	298.2	308.7	1408	46.3	3	0	No Failure
2	L	298.1	308.5	1498	49.4	5	0	No Failure
3	L	298.2	308.6	1433	39.5	7	0	No Failure
4	L	298.2	308.7	1408	40.0	9	0	No Failure

Skewness

Its values allows us to understand how data is distributed in relation to a symmetric or normal distribution.

Skewness ≈ 0: The distribution is approximately symmetrical. This indicates that the data is not biased toward the left or the right.

Skewness > 0 (positive skewness): The tail of the distribution is longer towards the right.

Skewness < 0 (negative skewness): The tail of the distribution is longer towards the left.

```
df.skew(axis=0, skipna=True, numeric_only=True)
```

	0
Air temperature [K]	0.114274
Process temperature [K]	0.015027
Rotational speed [rpm]	1.993171
Torque [Nm]	-0.009517
Tool wear [min]	0.027292
Target	5.151852

dtype: float64

Identifying categorical columns

```
cat_cols = df.select_dtypes(include='object').columns.tolist()
print(cat_cols)
```

```
['Type', 'Failure Type']
```

Value counts for categorical columns

```
for col in cat_cols:
    print(df[col].value_counts())
```

```
Type
L    6000
M    2997
H    1003
Name: count, dtype: int64
Failure Type
No Failure                 9652
Heat Dissipation Failure    112
Power Failure                95
Overstrain Failure           78
Tool Wear Failure            45
Random Failures              18
Name: count, dtype: int64
```

Identifying faulty machines with 'Target' value 1

```
df_f = df[df.Target == 1]
df_f['Failure Type'].value_counts()
```

Failure Type	count
Heat Dissipation Failure	112
Power Failure	95
Overstrain Failure	78
Tool Wear Failure	45
No Failure	9

dtype: int64

Removing rows with 'No Failure' in faulty machines

```
idx = df_f[df_f['Failure Type'] == 'No Failure'].index
df.drop(idx, axis=0, inplace=True)
```

Identifying non-faulty machines with 'Target' value 0

```
df_f = df[df.Target == 0]
df_f['Failure Type'].value_counts()
```

Failure Type	count
No Failure	9643
Random Failures	18

dtype: int64

Removing rows with 'Random Failures' in non-faulty machines

```
idx = df_f[df_f['Failure Type'] == 'Random Failures'].index
df.drop(idx, axis=0, inplace=True)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 9973 entries, 0 to 9999
Data columns (total 8 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Type                    9973 non-null   object
 1   Air temperature [K]     9973 non-null   float64
 2   Process temperature [K] 9973 non-null   float64
 3   Rotational speed [rpm]  9973 non-null   int64
 4   Torque [Nm]             9973 non-null   float64
 5   Tool wear [min]         9973 non-null   int64
 6   Target                  9973 non-null   int64
 7   Failure Type            9973 non-null   object
```

```
dtypes: float64(3), int64(3), object(2)
memory usage: 701.2+ KB
```

Replacing infinite values with NaN

```
df.replace([np.inf, -np.inf], np.nan, inplace=True)
```

```
columns_to_check = ['Type', 'Air temperature [K]', 'Process temperature [K]', 'Rotational speed [rpm]', 'Torque [Nm]', 'Tool wear [mi

for col in columns_to_check:
    print(df[df[col].isna()])
```

```
Empty DataFrame
Columns: [Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], Target, Failure Type]
Index: []
Empty DataFrame
Columns: [Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], Target, Failure Type]
Index: []
Empty DataFrame
Columns: [Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], Target, Failure Type]
Index: []
Empty DataFrame
Columns: [Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], Target, Failure Type]
Index: []
Empty DataFrame
Columns: [Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], Target, Failure Type]
Index: []
Empty DataFrame
Columns: [Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], Target, Failure Type]
Index: []
Empty DataFrame
Columns: [Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], Target, Failure Type]
Index: []
Empty DataFrame
Columns: [Type, Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], Target, Failure Type]
Index: []
```

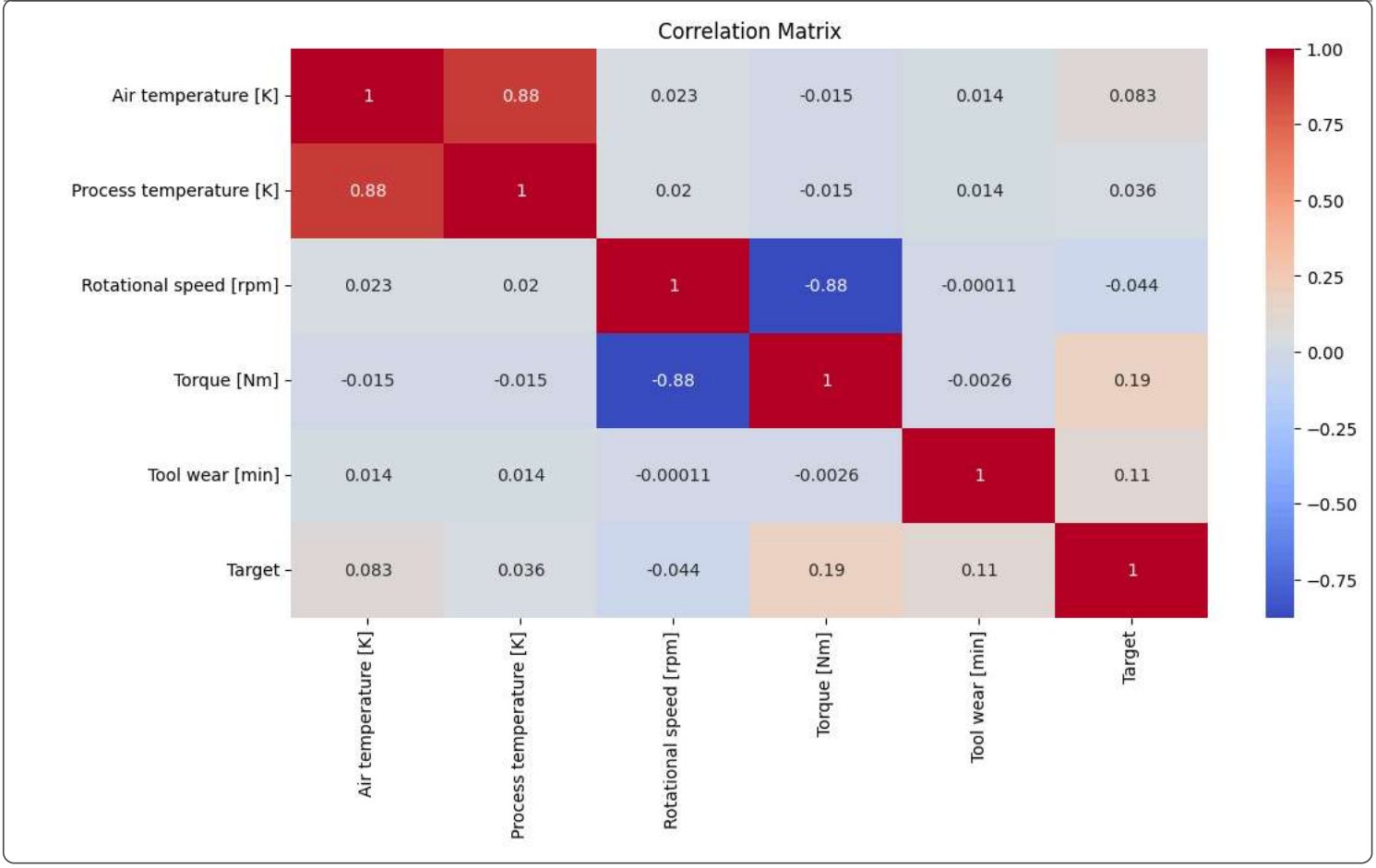
Correlation matrix

```
df.corr(method='pearson', numeric_only=True)
```

	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Target
Air temperature [K]	1.000000	0.876068	0.023332	-0.014553	0.014370	0.083066
Process temperature [K]	0.876068	1.000000	0.020200	-0.015275	0.013970	0.035997
Rotational speed [rpm]	0.023332	0.020200	1.000000	-0.875069	-0.000110	-0.044042
Torque [Nm]	-0.014553	-0.015275	-0.875069	1.000000	-0.002552	0.193420
Tool wear [min]	0.014370	0.013970	-0.000110	-0.002552	1.000000	0.106337
Target	0.083066	0.035997	-0.044042	0.193420	0.106337	1.000000

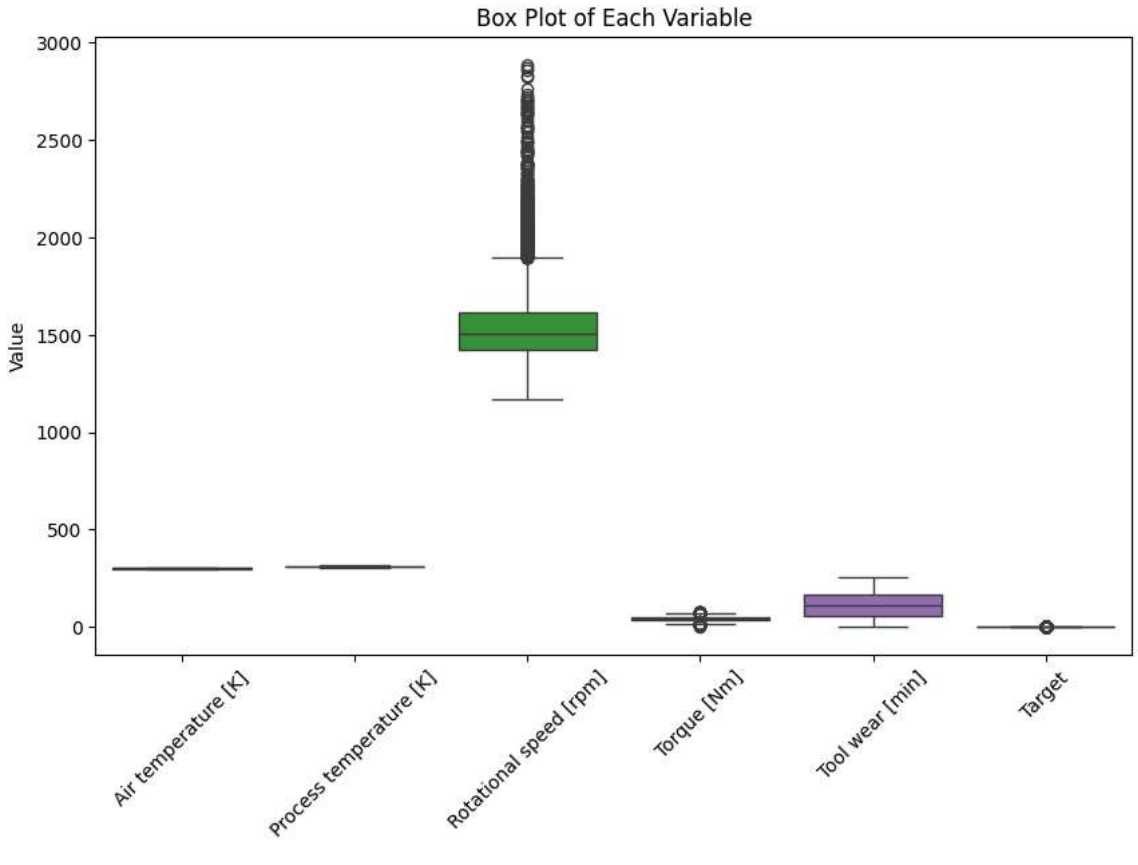
```
numeric_df = df.select_dtypes(include=[np.number])
numeric_df.replace([np.inf, -np.inf], np.nan, inplace=True)

plt.figure(figsize=(12, 6))
sns.heatmap(numeric_df.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```



Box plot

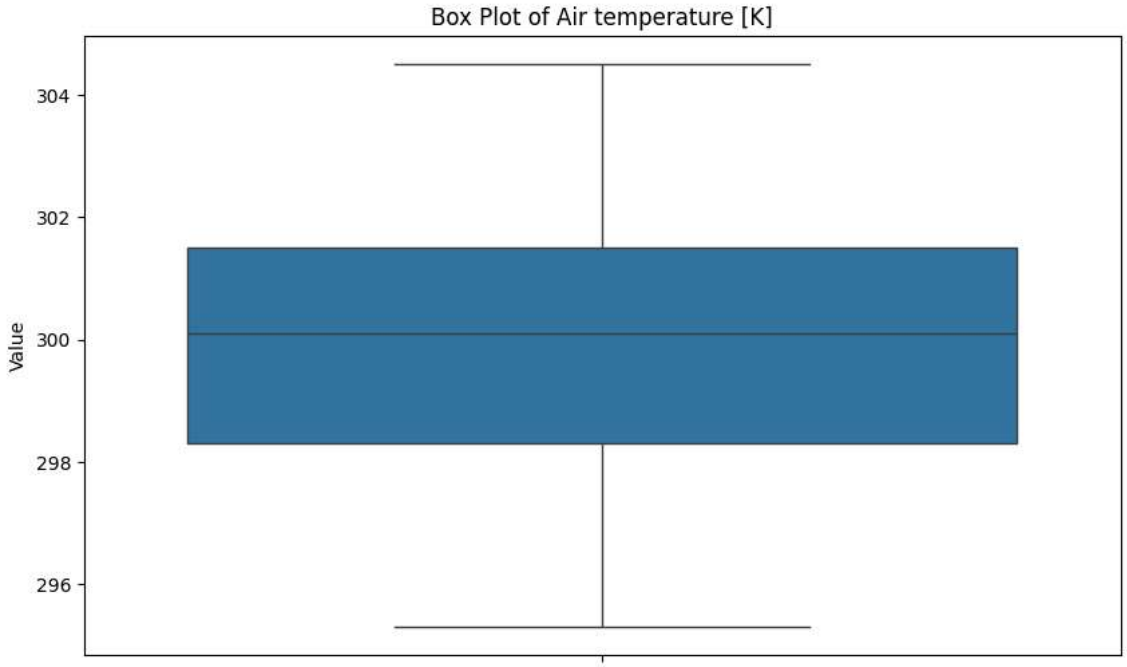
```
plt.figure(figsize=(10, 6))
sns.boxplot(data=df)
plt.title("Box Plot of Each Variable")
plt.xticks(rotation=45)
plt.ylabel("Value")
plt.show()
```

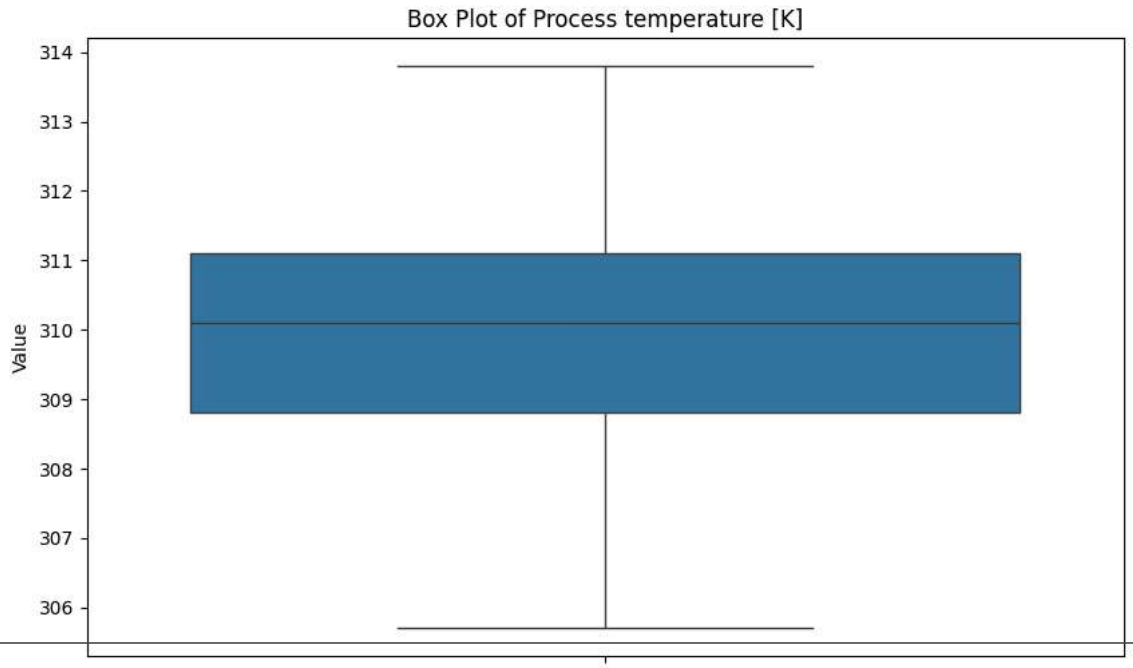


Individual box plot

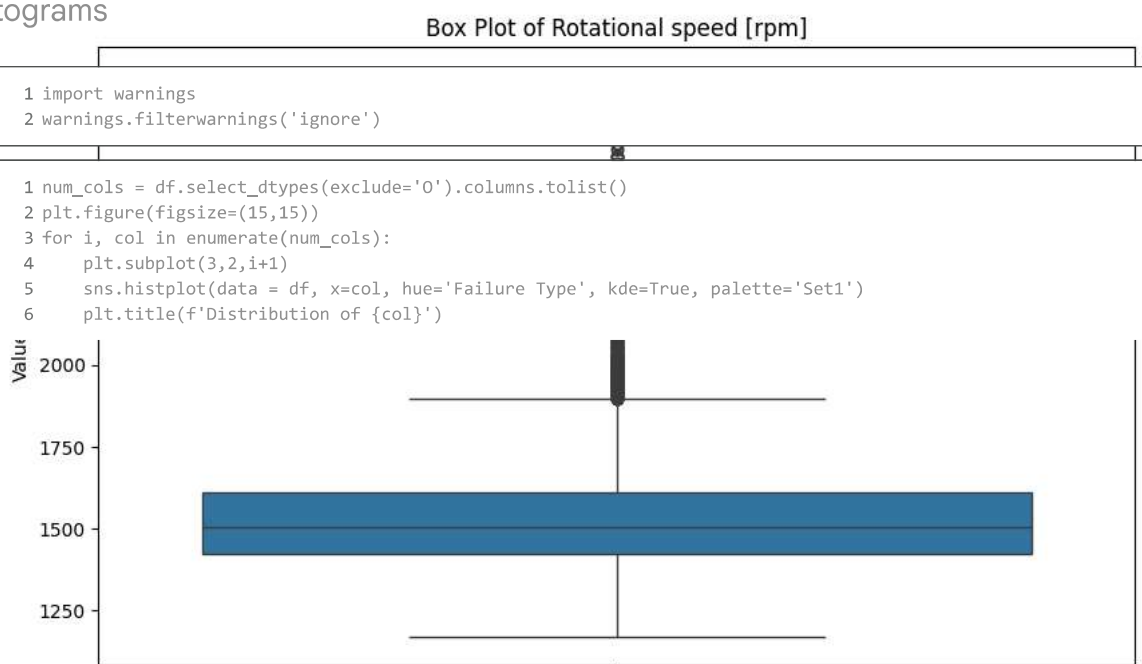
```
num_cols = df.select_dtypes(include='number').columns

for column in num_cols:
    plt.figure(figsize=(10, 6))
    sns.boxplot(data=df[column])
    plt.title(f"Box Plot of {column}")
    plt.ylabel("Value")
    plt.show()
```



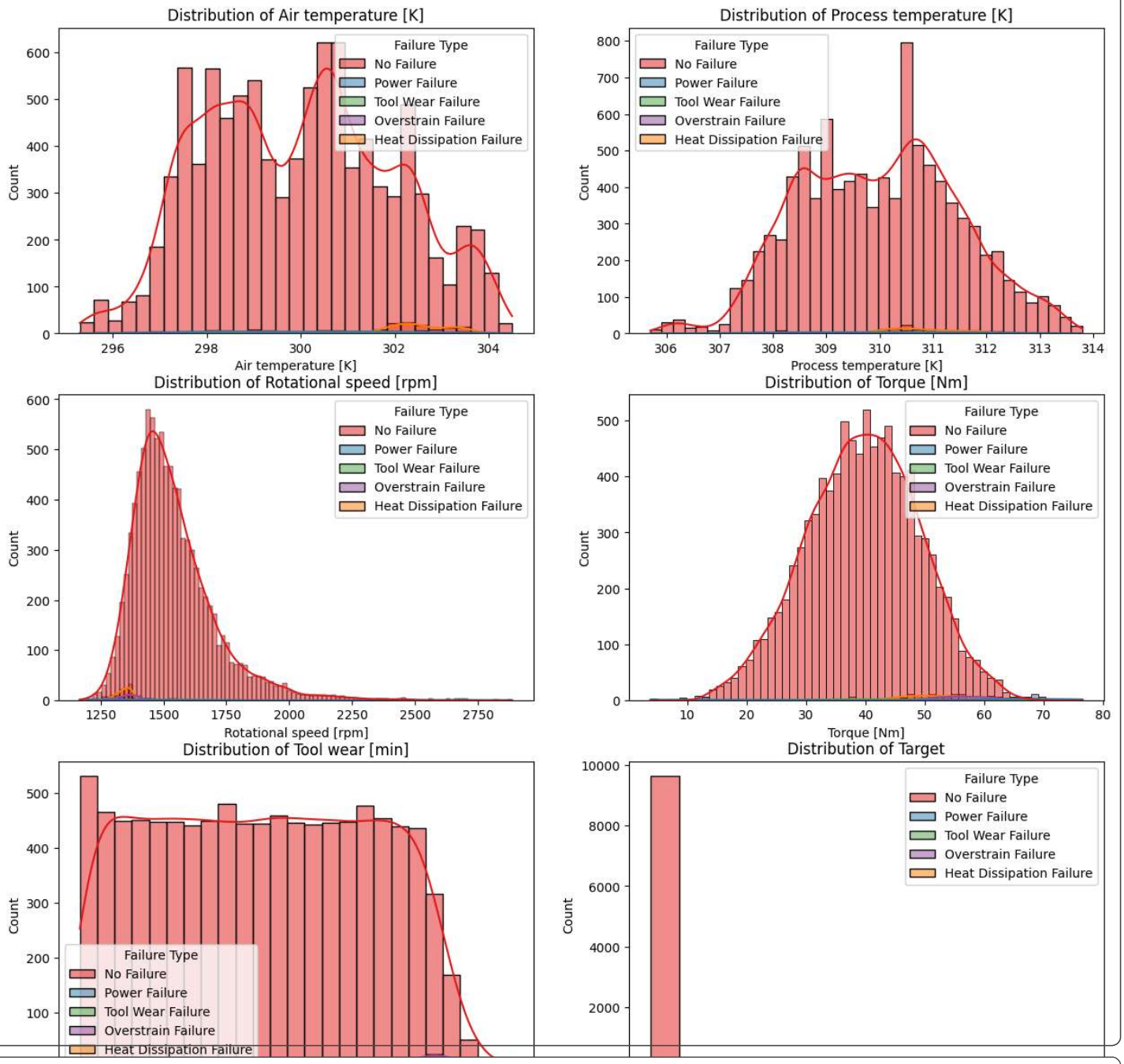


Histograms




```
import warnings
warnings.filterwarnings('ignore')
```

```
num_cols = df.select_dtypes(exclude='O').columns.tolist()
plt.figure(figsize=(15,15))
for i, col in enumerate(num_cols):
    plt.subplot(3,2,i+1)
    sns.histplot(data = df, x=col, hue='Failure Type', kde=True, palette='Set1')
    plt.title(f'Distribution of {col}')
```




```python
from scipy.stats import skew

# Extract continuous features
continuous_features = ['Air temperature [K]', 'Process temperature [K]', 'Rotational speed [rpm]', 'Torque [Nm]', 'Tool wear [min]']

# Generate normal distribution plots for each continuous feature
for feature in continuous_features:
    # Generate synthetic data for the feature (replace df[feature] with your actual data)
    data = df[feature]

    # Calculate skewness
    data_skewness = skew(data)

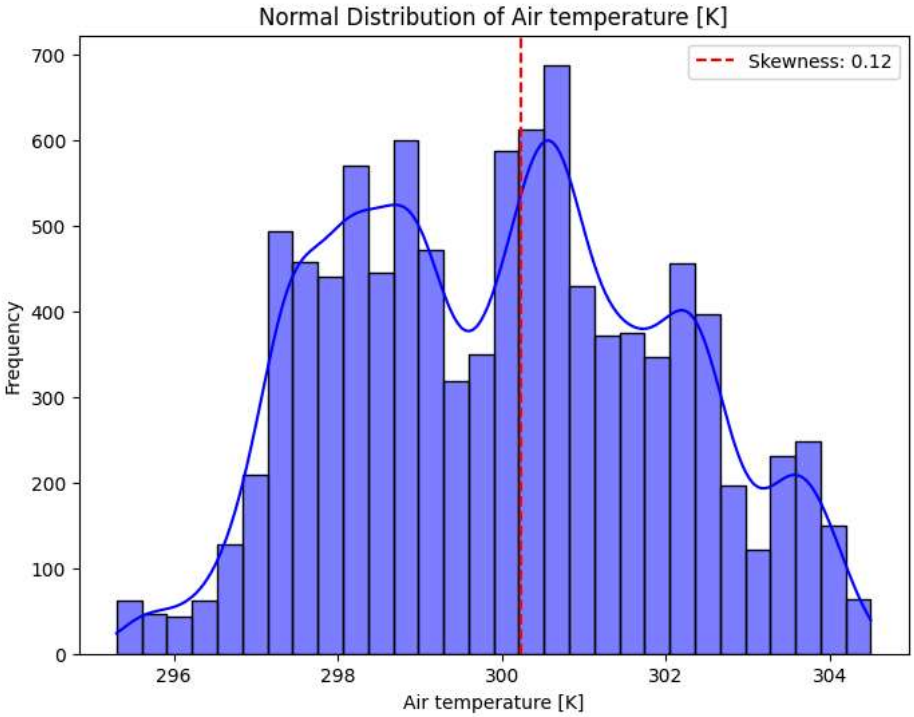
    # Create figure and axes
    plt.figure(figsize=(8, 6))

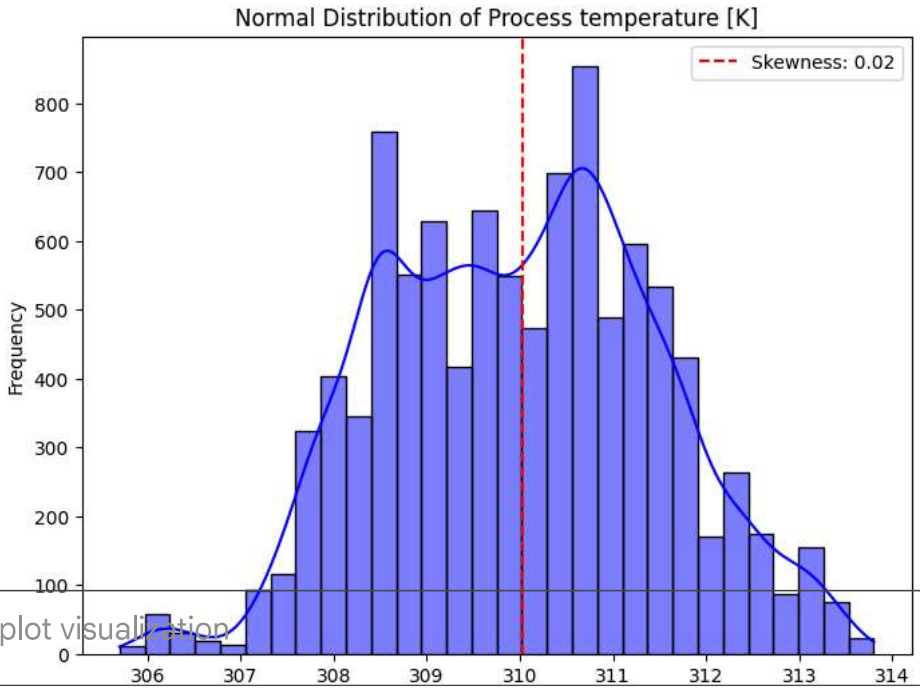
    # Plot histogram with kernel density estimation
    sns.histplot(data, kde=True, color='blue', bins=30)

    # Add vertical line to indicate skewness
    plt.axvline(x=data.mean() + data_skewness * data.std(), color='red', linestyle='--', label=f'Skewness: {data_skewness:.2f}')

    # Add labels and title
    plt.xlabel(feature)
    plt.ylabel('Frequency')
    plt.title(f'Normal Distribution of {feature}')
    plt.legend()

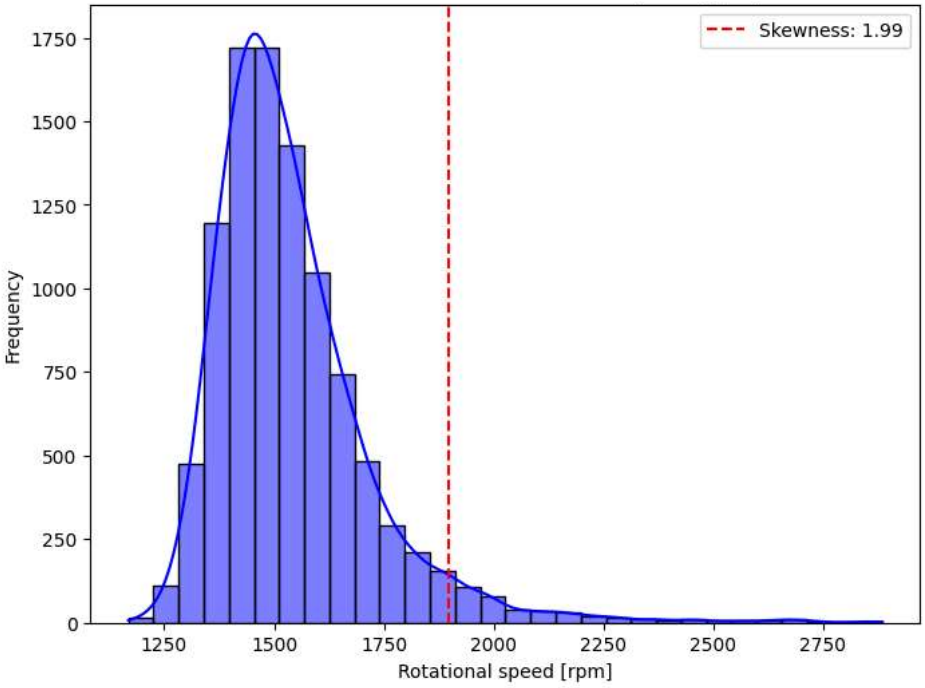
    # Show plot
    plt.show()
```



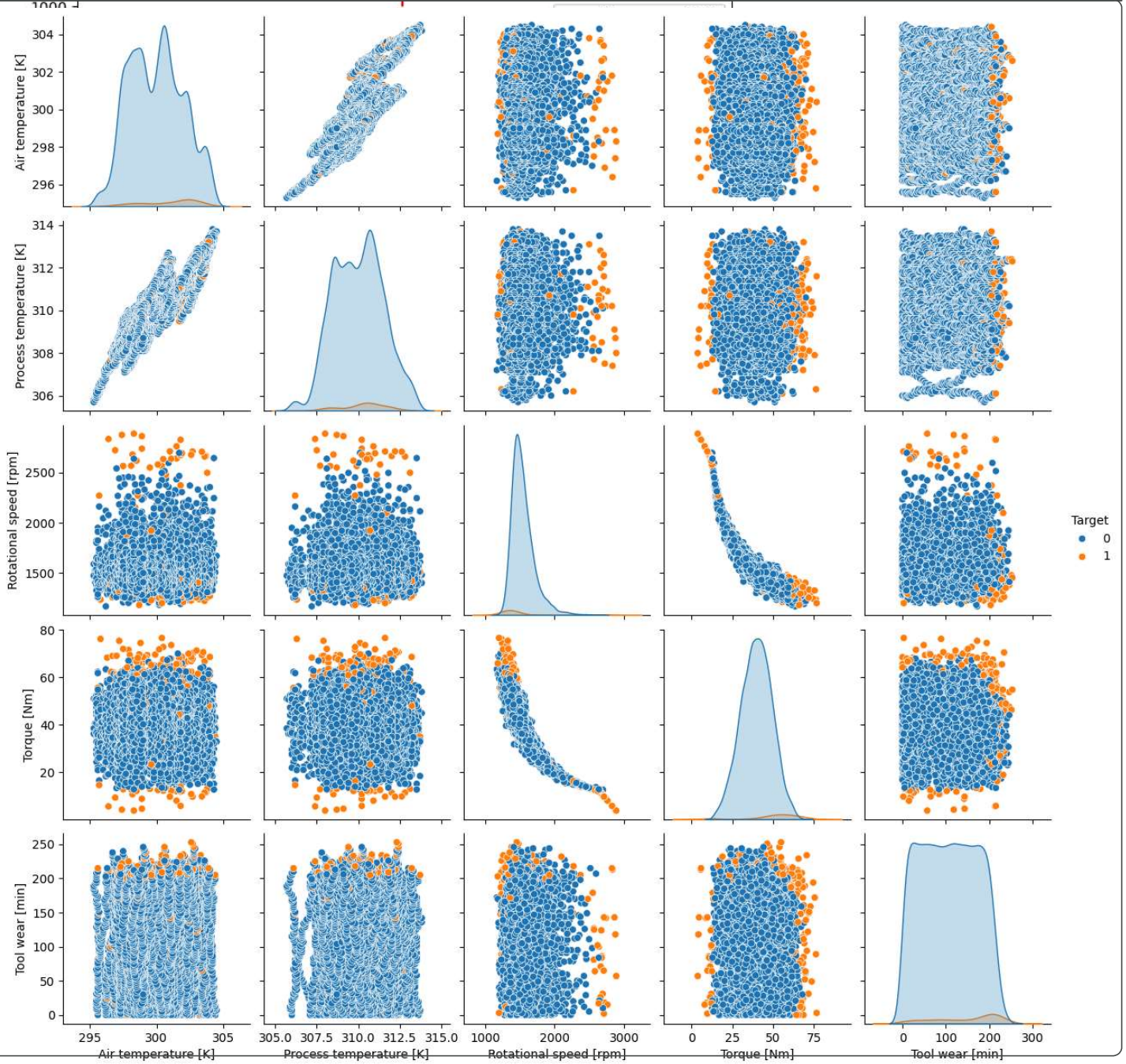


Pairplot visualization

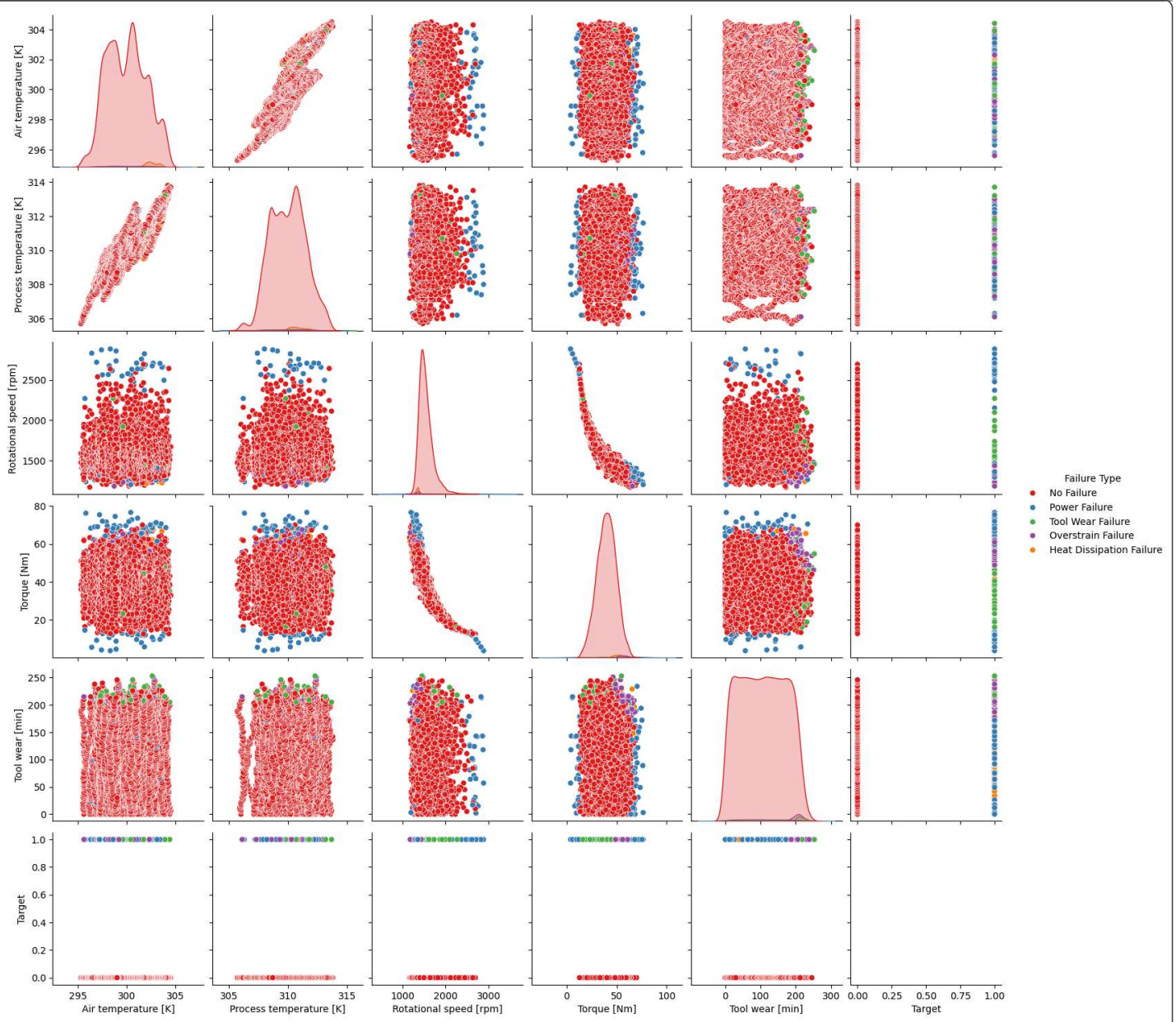
```
sns.pairplot(df, hue='Target')
plt.show()
```








```
1 sns.pairplot(df, hue='Failure Type', palette="Set1")
2 plt.show()
```



Value counts for categorical columns

```
1 for col in cat_cols:
2     print(df[col].value_counts())
```

```
Type
L    5984
M    2991
H     998
Name: count, dtype: int64
Failure Type
No Failure                 9643
Heat Dissipation Failure    112
Power Failure                95
Overstrain Failure           78
Tool Wear Failure            45
Name: count, dtype: int64
```

```
# Variables
categorical_var = 'Failure Type'
continuous_vars = df.select_dtypes(include=['float64', 'int64']).columns.tolist()

# Subplots
num_vars = len(continuous_vars)
num_cols = 3  # plots per line
num_rows = (num_vars + num_cols - 1) // num_cols  # neccesary number of rows

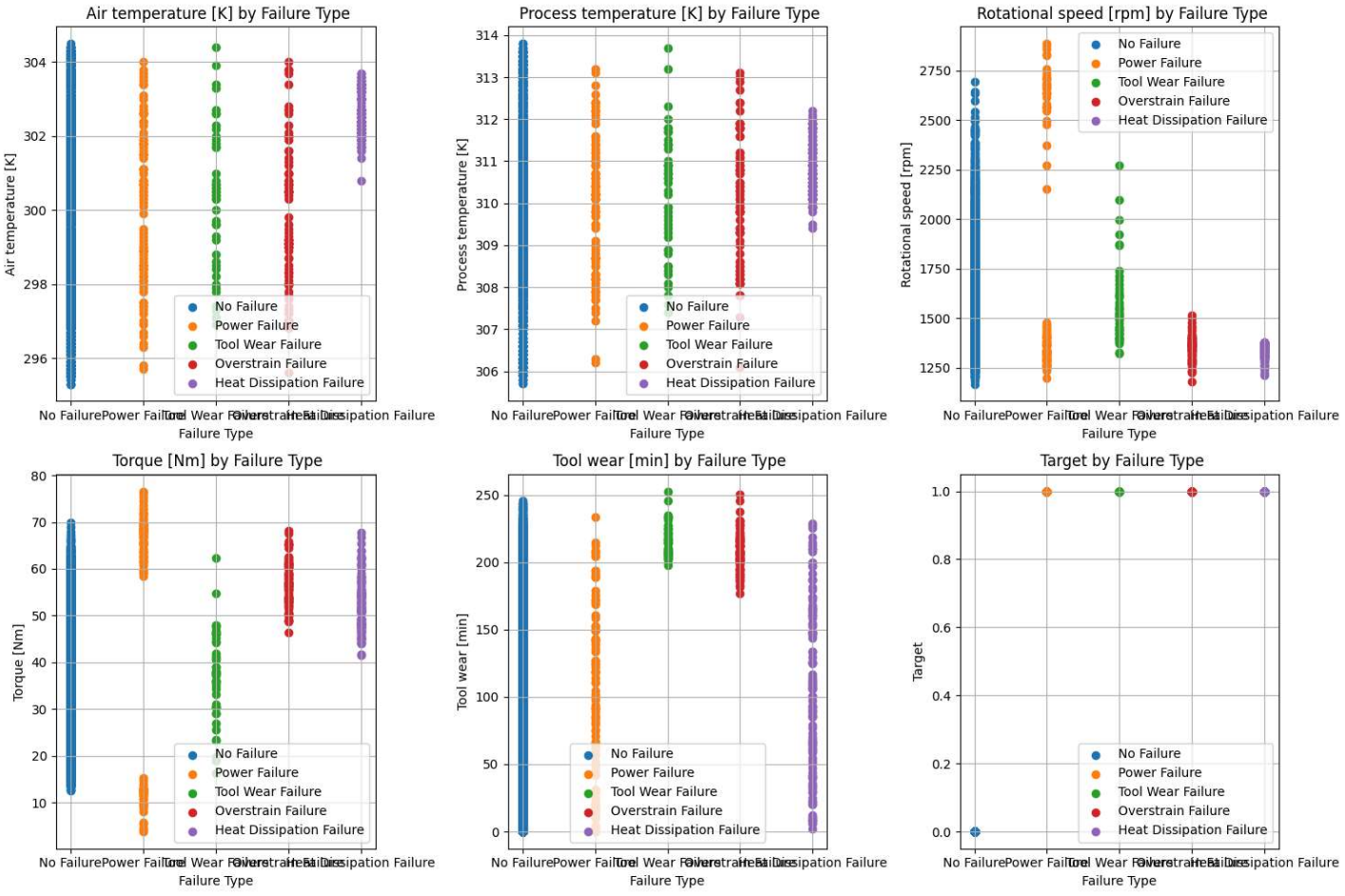
fig, axes = plt.subplots(num_rows, num_cols, figsize=(15, 5 * num_rows))
axes = axes.flatten()  # flatting the matrix axis

# Plot per variable
for i, var in enumerate(continuous_vars):
    for failure_type in df[categorical_var].unique():
        subset = df[df[categorical_var] == failure_type]
        axes[i].scatter(subset[categorical_var], subset[var], label=failure_type)

    axes[i].set_title(f' {var} by {categorical_var}')
    axes[i].set_xlabel(categorical_var)
    axes[i].set_ylabel(var)
    axes[i].legend()
    axes[i].grid()

# Deleting empty axis if there are less varibles than subplots
for j in range(i + 1, num_rows * num_cols):
    fig.delaxes(axes[j])

plt.tight_layout()  # Adjustment of layout not to shorten the tags
plt.show()
```




```
# Identify the categorical variable and the variable to plot
categorical_var_x = 'Failure Type'
```

```
categorical_var_y = 'Type'
# Create a figure for a single plot
plt.figure(figsize=(10, 6))

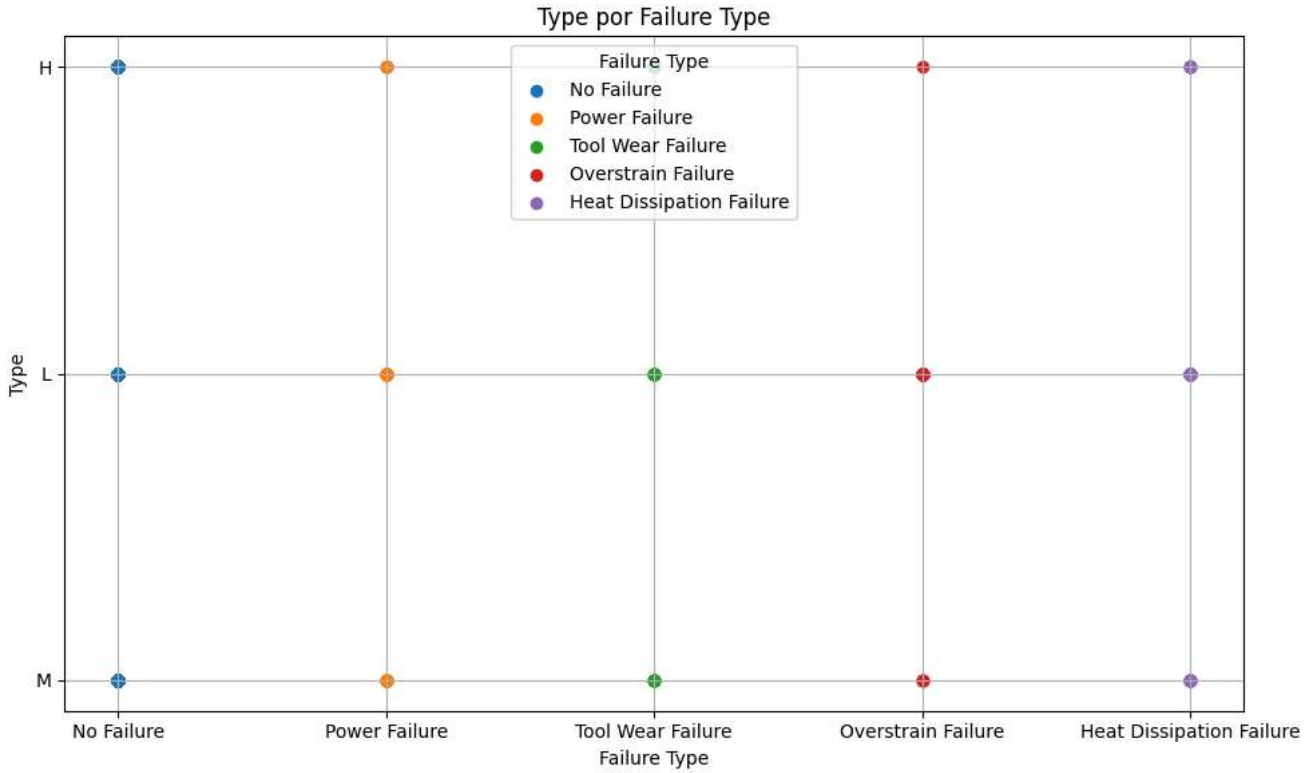
# Graphing
for failure_type in df[categorical_var_x].unique():
    subset = df[df[categorical_var_x] == failure_type]
    plt.scatter(subset[categorical_var_x], subset[categorical_var_y], label=failure_type)

# Graph configuration
plt.title(f'{categorical_var_y} por {categorical_var_x}')
plt.xlabel(categorical_var_x)
plt.ylabel(categorical_var_y)
plt.legend(title=categorical_var_x)
plt.grid()

plt.tight_layout()
plt.show()

# Create a contingency table
contingency_table = pd.crosstab(df['Type'], df['Failure Type'], margins=True, margins_name="Total")

# Show the table
print(contingency_table)
```

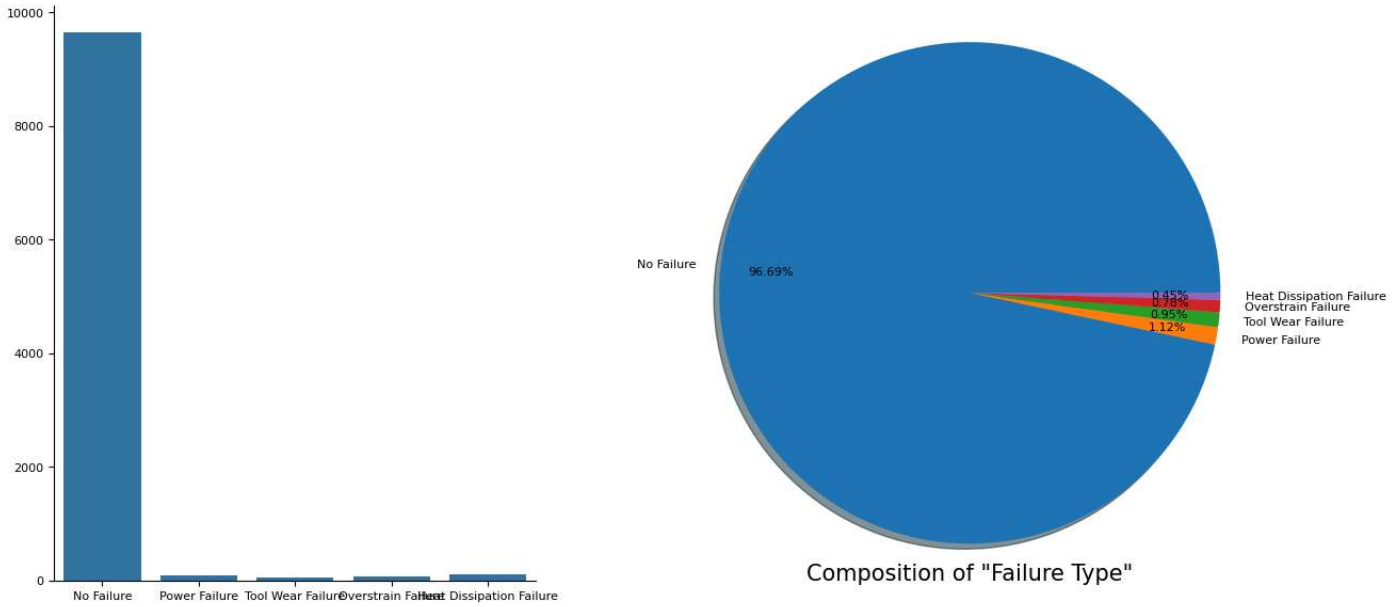



```
Failure Type  Heat Dissipation Failure  No Failure  Overstrain Failure  \
Type
H                                   8         978                   1
L                                  74        5753                  73
M                                  30        2912                   4
Total                             112        9643                  78

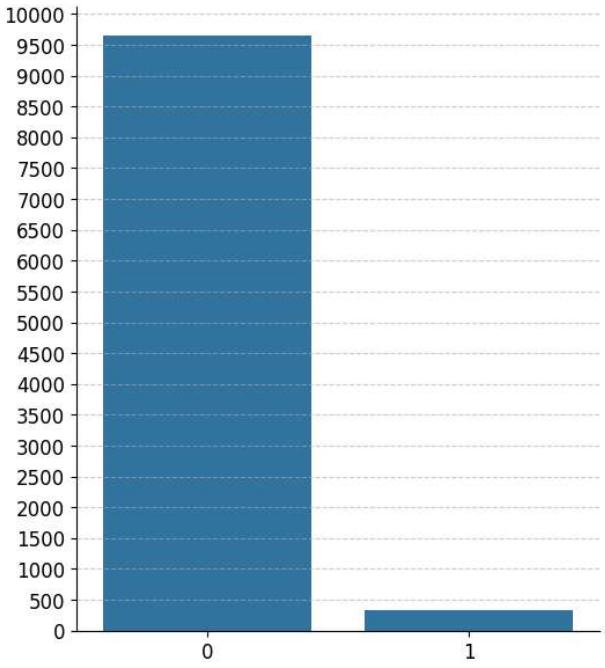
Failure Type  Power Failure  Tool Wear Failure  Total
Type
H                         5                  6    998
L                        59                 25   5984
M                        31                 14   2991
Total                    95                 45   9973
```

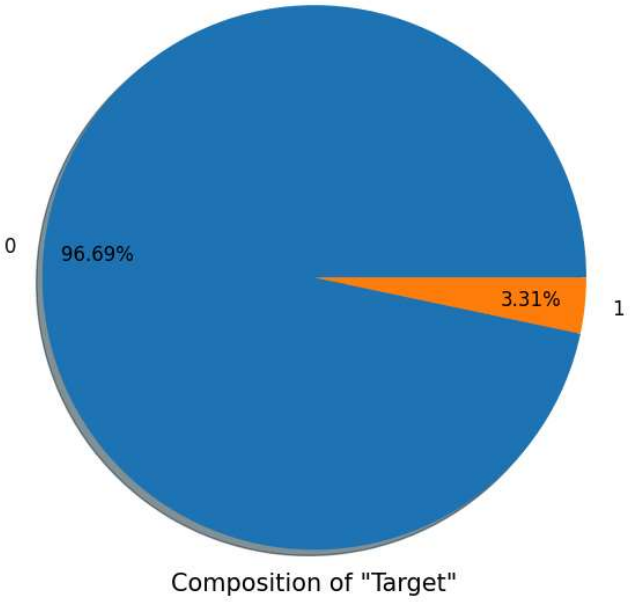
```
# defining the target variable
df['Failure Type'].value_counts()
# checking the target
plt.figure(figsize=(14,7))
# barplot
ax1 = plt.subplot(1,2,1)
cp = sns.countplot(x=df["Failure Type"])
ax1.set_xlabel(" ")
ax1.set_ylabel(" ")
plt.xticks(fontsize=8)
plt.yticks(fontsize=8)
sns.despine(top=True, right=True)
# pieplot
ax2 = plt.subplot(1,2,2)
plt.pie(df["Failure Type"].value_counts(),
        labels=list(df["Failure Type"].unique()),
        autopct='%1.2f%%',
        pctdistance=0.8,
        shadow=True,
```

```
        radius=1.3,
        textprops={'fontsize':8}
       )
ax2.set_xlabel(" ")
plt.xlabel('Composition of "Failure Type"', fontsize=15, labelpad=20)
plt.subplots_adjust(wspace=0.4)
plt.show()
```

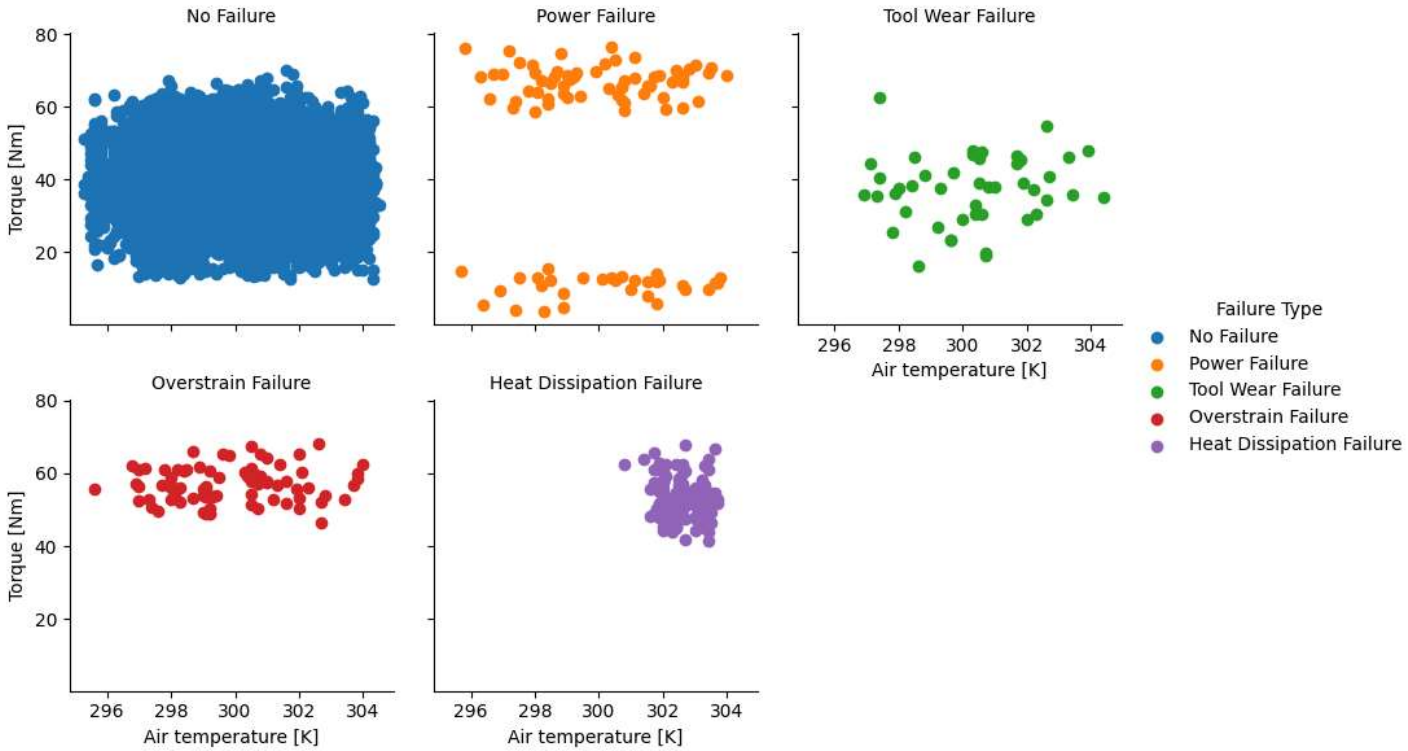



```
# defining the target variable
df['Target'].value_counts()
# checking the target
plt.figure(figsize=(14,7))
# barplot
ax1 = plt.subplot(1,2,1)
cp = sns.countplot(x=df["Target"])
ax1.set_xlabel(" ")
ax1.set_ylabel(" ")
plt.xticks(fontsize=12)
plt.yticks(np.arange(0, 10001, 500), fontsize=12)
plt.grid(axis="y", linestyle="--", alpha=0.6)
sns.despine(top=True, right=True)
# pieplot
ax2 = plt.subplot(1,2,2)
plt.pie(df["Target"].value_counts(),
        labels=list(df["Target"].unique()),
        autopct='%1.2f%%',
        pctdistance=0.8,
        shadow=True,
        radius=1.3,
        textprops={'fontsize':12}
       )
ax2.set_xlabel(" ")
plt.xlabel('Composition of "Target"', fontsize=15, labelpad=20)
plt.subplots_adjust(wspace=0.4)
plt.show()
```






```
# Create a FacetGrid with seaborn
g = sns.FacetGrid(df, col="Failure Type", hue="Failure Type", col_wrap=3)
# Map a scatter plot to each facet
g.map(plt.scatter, "Air temperature [K]", "Torque [Nm]")
# Add a title to each facet
g.set_titles(col_template="{col_name}")
# Add legend
g.add_legend()
# Show the plot
plt.show()
```



Encoding and mapping categorical variables

```
# Importing labelEncoder library
from sklearn.preprocessing import LabelEncoder

# Columns to encode
columns_to_encode = ['Type', 'Failure Type']

# Dictionary to hold the mappings for each column
```

```
label_encoders = {}
mappings = {}

# Apply LabelEncoder to each column
for column in columns_to_encode:
    encoder = LabelEncoder()
    df[column] = encoder.fit_transform(df[column])
    label_encoders[column] = encoder
    mappings[column] = dict(zip(encoder.classes_, encoder.transform(encoder.classes_)))

    # Display the mappings
for column, mapping in mappings.items():
    print(f'Mapping for {column}: {mapping}')
```

```
Mapping for Type: {'H': np.int64(0), 'L': np.int64(1), 'M': np.int64(2)}
Mapping for Failure Type: {'Heat Dissipation Failure': np.int64(0), 'No Failure': np.int64(1), 'Overstrain Failure': np.int64(2), 'Power F
```

```
# Rename the variables
df.rename(columns={'Air temperature [K]': 'Air_temperature', 'Rotational speed [rpm]': 'Rotational_speed', 'Torque [Nm]': 'Torque','T
```

```
# Define features (X) and target variable (y)
X = df.drop(['Target', 'Failure_Type'], axis=1)
y = df['Target']

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
# Models to evaluate
models = {
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "GradientBoosting": GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42),
    "Logistic Regressor": LogisticRegression(max_iter=500),
    "SVM": SVC(probability=True),
    "Nearest Neighbors": KNeighborsClassifier(),
    "XGBoost": XGBClassifier(use_label_encoder=False, eval_metric="logloss"),
    "Tree Decision": DecisionTreeClassifier()
}

# Define hyperparameters for each model
param_grid = {
    "Random Forest": {
        "n_estimators": [50, 100, 200],
        "max_depth": [None, 10, 20],
        "min_samples_split": [2, 5, 10]
    },
    "GradientBoosting": {
        "n_estimators": [50, 100, 200],
        "learning_rate": [0.05, 0.1, 0.2],
        "max_depth": [3, 5, 10]
    },
    "Logistic Regressor": {
        "C": [0.1, 1, 10],
        "solver": ["liblinear", "lbfgs"]
    },
    "SVM": {
        "C": [0.1, 1, 10],
        "kernel": ["linear", "rbf"]
    },
    "Nearest Neighbors": {
        "n_neighbors": [3, 5, 10],
        "weights": ["uniform", "distance"]
    },
    "XGBoost": {
        "n_estimators": [50, 100, 200],
        "learning_rate": [0.05, 0.1, 0.2],
        "max_depth": [3, 5, 10]
    },
    "Tree Decision": {
        "max_depth": [None, 5, 10],
        "min_samples_split": [2, 5, 10]
    }
}

# Perform GridSearch for each model
best_models = {}

for name, model in models.items():
    if name in param_grid:
        grid_search = GridSearchCV(model, param_grid[name], cv=3, scoring="accuracy", n_jobs=-1)
        grid_search.fit(X, y)
        best_models[name] = grid_search.best_estimator_

        print(f"\n🔷 Best configuration for {name}:")
        print(grid_search.best_params_)
```

```
    else:
```

```
🔷 Best configuration for Random Forest:
{'max_depth': 10, 'min_samples_split': 5, 'n_estimators': 50}

🔷 Best configuration for GradientBoosting:
{'learning_rate': 0.05, 'max_depth': 5, 'n_estimators': 50}

🔷 Best configuration for Logistic Regressor:
{'C': 0.1, 'solver': 'liblinear'}

🔷 Best configuration for SVM:
{'C': 10, 'kernel': 'rbf'}

🔷 Best configuration for Nearest Neighbors:
{'n_neighbors': 10, 'weights': 'distance'}

🔷 Best configuration for XGBoost:
{'learning_rate': 0.05, 'max_depth': 10, 'n_estimators': 50}

🔷 Best configuration for Tree Decision:
{'max_depth': 10, 'min_samples_split': 10}
```

```
# Configure K-Fold Cross Validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# Models to evaluate
models = {
    "Random Forest": RandomForestClassifier(max_depth= 10, min_samples_split= 5, n_estimators= 50, random_state=42),
    "GradientBoosting": GradientBoostingClassifier(n_estimators=50, learning_rate=0.05, max_depth=5, random_state=42),
    "Logistic Regressor": LogisticRegression(max_iter=500, C= 0.1, solver="liblinear"),
```